

ESP32 RELAY NODE FOR BITCHAT

A mains-powered box that sits in a window and quietly extends a Bluetooth mesh. It pretends to be a phone running bitchat, listens for packets, subtracts one from a single byte, and passes them on. That's the entire job — and it's the difference between a mesh that dies at the end of your street and one that covers the block.

SKILL — BEGINNER, NO SOLDERING REQUIRED **COST** — ~\$15-30 USD

WRITTEN FOR NIMBLE-ARDUINO 2.X

CONTENTS

01 What you're building, and what it can't do

02 Parts list

03 Wiring

04 Install the software

05 Raise the connection limit

06 Load the code

07 Verify against a real phone

08 Deploy it

09 When it doesn't work

10 Power, antennas and the law

00

WHAT YOU'RE BUILDING

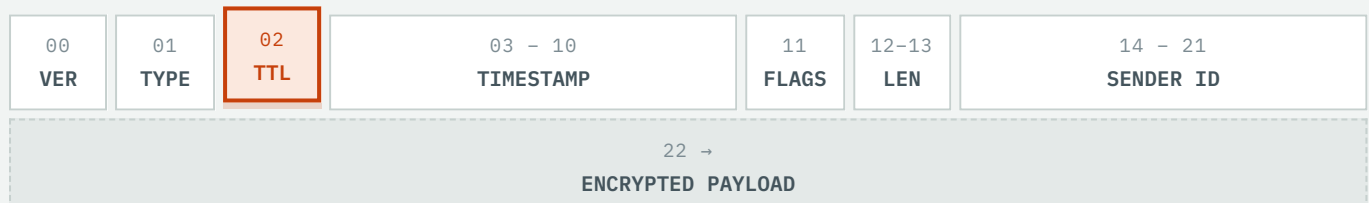
Read this part. It will save you from a confusing afternoon.

bitchat is a messaging app that works without internet. Phones talk directly to each other over Bluetooth Low Energy, and every phone also acts as a **relay** — when it hears a message that isn't for it, it passes it along. Each hop covers maybe 10–30 metres, and a message is allowed seven hops before it expires.

The weakness is obvious: it only works if there are enough phones in between. Phones move, sleep, and run out of battery. A relay node is a cheap chip that does the relaying job permanently, from a fixed spot, on wall power, with a better antenna than a phone has.

THE ONE BYTE THAT MATTERS

Every bitchat packet starts with a fixed 14-byte header. A relay doesn't read the message — messages are encrypted and the relay has no keys. It only reads far enough to make a routing decision, and it only ever *writes* one byte:



Byte 02 is the time-to-live. The relay decrements it and re-sends the packet byte-for-byte otherwise untouched. bitchat deliberately computes its signatures *excluding* the TTL byte so relays can do exactly this without breaking them.

WHY THIS DESIGN IS SAFE TO BUILD

The relay never decrypts, never verifies signatures, and never reassembles fragmented messages. It's a dumb pipe. That means a compromised or buggy relay leaks nothing — it can drop your traffic, but it can't read it.

HONEST EXPECTATIONS

- **This is not a walkie-talkie.** A relay only helps if there are bitchat users near it. One relay alone does nothing.

- **Range is line-of-sight-ish.** A relay in a second-floor window facing a street will vastly outperform one on a desk indoors. Height and clear sightlines beat transmit power almost every time.
- **Kilometre range needs a different radio.** Bluetooth won't do it. If you need that, look at bridging bitchat to LoRa/Meshtastic — a much larger project than this one.
- **The app can change.** bitchat is actively developed. If a future update changes the packet layout or service UUID, your relay goes deaf until you update it. Section 07 shows how to check.

01 PARTS LIST

Two items are genuinely required. Everything else is comfort.

PART	WHAT TO ACTUALLY SEARCH FOR	NOTES
REQUIRED ESP32 board with external antenna socket	ESP32-DevKitC-32UE or ESP32-WROOM-32U dev board	The U is the whole point — it means a U.FL/IPEX antenna socket instead of a printed-on antenna. A board without it works, but gives you little range advantage over a phone.
REQUIRED 2.4 GHz antenna	2.4GHz 6dBi antenna U.FL IPEX pigtail	Many "U" boards ship with a small one included. A bigger external one is the single cheapest range upgrade you can make.
REQUIRED USB data cable	Micro-USB or USB-C, matching your board	Must carry data, not just power. See the trap below.
REQUIRED 5V USB power supply	Any phone charger, 1A or more	For permanent deployment once flashing is done.
OPTIONAL SSD1306 OLED display	0.96" OLED I2C SSD1306 128x64	Shows link count at a glance so you don't need a laptop to check on it.
OPTIONAL Female-to-female jumper wires	dupont female to female jumper	Only if fitting the OLED. Four wires. No soldering.

	PART	WHAT TO ACTUALLY SEARCH FOR	NOTES
OPTIONAL	Weatherproof enclosure + cable gland	IP65 junction box , PG7 cable gland	Only if mounting outdoors. Keep the antenna outside the box.

CLASSIC BEGINNER TRAP #1 – THE CHARGE-ONLY CABLE

Cheap USB cables bundled with power banks often have no data wires. Your board will light up but never appear as a serial port, and you'll spend an hour reinstalling drivers. If no port shows up,

TRY A DIFFERENT CABLE FIRST.

CLASSIC BEGINNER TRAP #2 – SMA VS RP-SMA

These connectors look identical and do not mate properly. SMA has a pin in the centre of the male side; RP-SMA has a hole. Buy your pigtail and your antenna as a matched pair, from the same listing if you can.

02 WIRING

The honest answer: for the basic build, there is none.

A relay is a self-contained radio. The minimum build is three physical actions: clip on the antenna, plug in USB, done. There are no components to connect and nothing to solder.

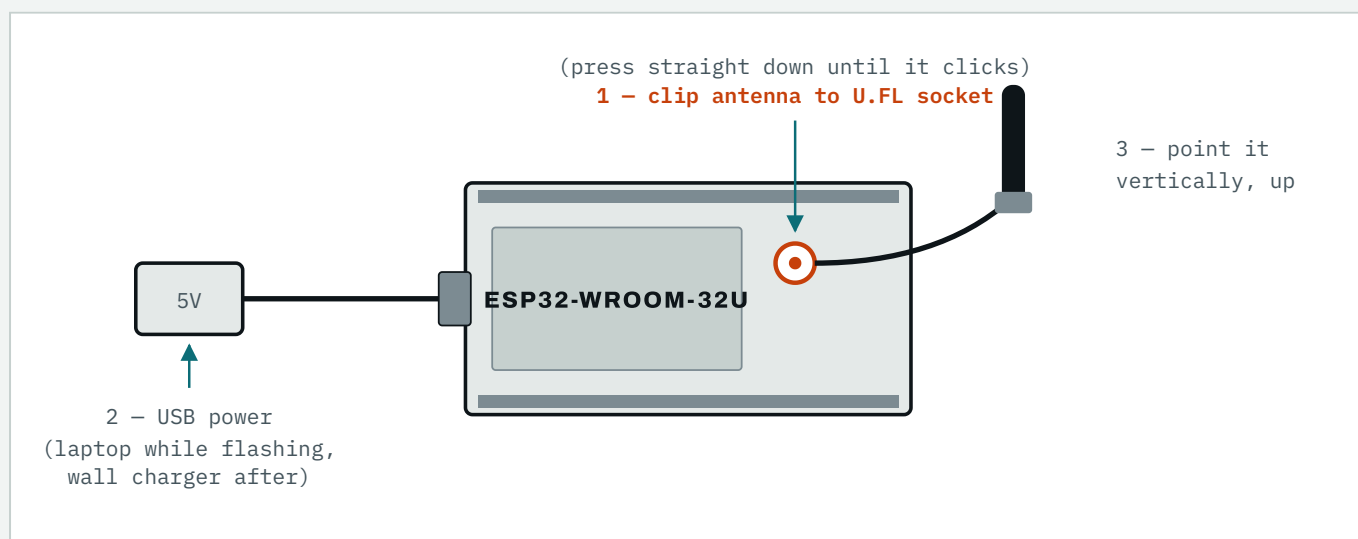


Figure 1 – Minimum build. The U.FL connector is fragile: press it on straight down with a fingernail or the flat of a small screwdriver until it clicks. Never twist it, and never yank the cable to remove it.

OPTIONAL: THE STATUS DISPLAY

If you want to glance at the relay and see how many peers it's talking to, add an SSD1306 OLED. It's four wires, all female-to-female jumpers pushed onto header pins — no soldering if your board came with headers pre-fitted.

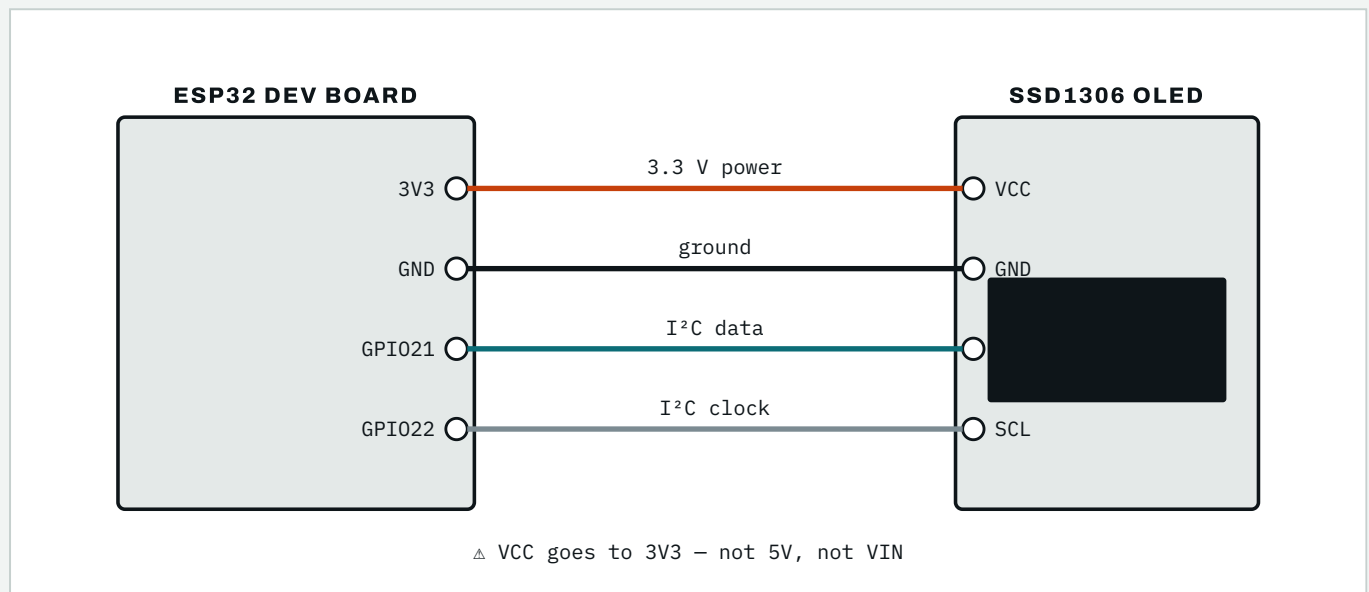


Figure 2 – Optional OLED wiring for a classic ESP32. On an **ESP32-S3** the default I²C pins are usually GPIO8 (SDA) and GPIO9 (SCL) instead – check your board's pinout card. The display code isn't included in the sketch below; add it once the relay itself is confirmed working.

3V3, NOT 5V

Most SSD1306 modules tolerate 5V, but the ESP32's GPIO pins are 3.3V logic. Powering the display from 3V3 keeps everything at one voltage and removes any risk to the chip. If you're unsure, use 3V3.

03 INSTALL THE SOFTWARE

About fifteen minutes, most of it downloading.

1. Install the Arduino IDE

Get Arduino IDE 2.x from arduino.cc/en/software and install it normally.

2. Add ESP32 board support

1. Open `File → Preferences`.
2. In **Additional boards manager URLs**, paste:
`https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. Click OK, then open `Tools → Board → Boards Manager`.
4. Search `esp32`, install **esp32 by Espressif Systems**. It's a large download — let it finish completely.

3. Install the NimBLE library

1. Open `Tools → Manage Libraries`.
2. Search `NimBLE-Arduino`.
3. Install **NimBLE-Arduino by h2zero**, version **2.x** (2.5.0 or newer).

THE VERSION MATTERS — A LOT

The sketch in this guide is written for NimBLE-Arduino

2.x

. Version 1.x changed several callback signatures and will produce a wall of compiler errors. If you see errors mentioning `onResult`, `NimBLEAdvertisedDeviceCallbacks` or `setPower`, check your library version first — that's almost always the cause.

4. Select your board

- For an ESP32-WROOM-32U board: `Tools → Board → ESP32 Arduino → ESP32 Dev Module`
- For an ESP32-S3 board: `ESP32S3 Dev Module`
- Leave every other setting at its default. NimBLE is small; you don't need a custom partition scheme.

04 RAISE THE CONNECTION LIMIT

Skip this and your relay caps out at three peers.

NimBLE defaults to **3** simultaneous BLE connections. That's fine for a sensor, but a relay wants many links at once — it needs inbound connections from phones *and* outbound

connections to other peers simultaneously. Three is quickly exhausted. NimBLE supports up to nine on ESP32.

1. Find your Arduino libraries folder:

- **Windows** — `Documents\Arduino\libraries\NimBLE-Arduino\src\`
- **macOS** — `~/Documents/Arduino/libraries/NimBLE-Arduino/src/`
- **Linux** — `~/Arduino/libraries/NimBLE-Arduino/src/`

2. Open `nimconfig.h` in any text editor.

3. Search the file for `MAX_CONNECTIONS`.

4. Change its value from `3` to `9`, and make sure the line isn't commented out with `//`.

5. Save, and fully restart the Arduino IDE.

WHY "SEARCH FOR IT" RATHER THAN A COPY-PASTE LINE

The exact macro name has changed between NimBLE versions. Searching for `MAX_CONNECTIONS` finds the right line regardless of which version you installed — a copy-pasted define might silently not match anything.

Using PlatformIO instead? Add this to `platformio.ini` rather than editing library files:

PLATFORMIO.INI

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
lib_deps = h2zero/NimBLE-Arduino@^2.5.0
build_flags = -D CONFIG_BT_NIMBLE_MAX_CONNECTIONS=9
monitor_speed = 115200
```

05 LOAD THE CODE

One file. Paste, plug in, upload.

1. Open the file `bitchat_relay.ino` supplied alongside this guide. (Or make a new sketch and paste its contents in — Arduino will ask to create a folder of the same name, which is

fine.)

2. Connect the board by USB.
3. Pick the port: `Tools → Port` . On Windows it's a `COM` number; on macOS something like `/dev/cu.usbserial-...` ; on Linux `/dev/ttyUSB0` .
4. Press **Upload** (the arrow icon).
5. Open `Tools → Serial Monitor` and set the speed to **115200** in the dropdown, or you'll see only garbage characters.

CLASSIC BEGINNER TRAP #3 – "FAILED TO CONNECT... WAIT FOR DOWNLOAD MODE"

Many ESP32 boards need a nudge to enter flashing mode. When the upload reaches

`Connecting.....` ,

PRESS AND HOLD THE BOOT BUTTON

on the board until it starts writing, then release. If your board has no BOOT button, tap EN/RST once as the connecting dots appear.

WHAT A HEALTHY BOOT LOOKS LIKE

Within a few seconds of the upload finishing, the serial monitor should show:

```
=== bitchat relay booting ===  
[srv ] advertising as bitchat peer  
[scan] scanning for bitchat peers  
[stat] links=0 heap=213...
```

`links=0` is correct and expected at this point — there's nothing to talk to yet. If you see the three startup lines and a repeating `[stat]` heartbeat, the firmware is running properly.

06 VERIFY AGAINST A REAL PHONE

Do not skip this. This is the step that catches silent failure.

A relay with a wrong byte offset doesn't crash — it quietly corrupts or drops everything, and you won't find out until you need it. So before you mount it anywhere, confirm it's actually seeing real bitchat traffic.

The sketch ships with `STRICT_HEADER_LOGGING` set to `1`, which prints the raw header bytes of everything it receives.

1. Leave the relay plugged into your computer with the serial monitor open.
2. Install bitchat on a phone, open it, and stand next to the relay.
3. Send a public/broadcast message from the phone.
4. Watch the serial monitor.

What you're looking for

```
[srv ] peer connected to us, handle=1
[link] + handle=1 role=peripheral (total 1)
[srv ] MTU now 517 on handle 1
[rx ] handle=1 len=76 hdr=01 04 07 00 00 01 93 6F ... | ver=1 type=4 ttl=7
[relay] forwarded 76 byte packet to 0 link(s)
```

Check each of these three things:

- **ver=1** — the protocol version parsed as 1. If this reads something wild like 147, your offsets are wrong or the app changed format.
- **ttl=7** — a freshly sent packet starts at TTL 7. Values of 7, 6, 5 are all plausible. A value of 200+ means you're misreading byte 2.
- **forwarded ... to 0 link(s)** — correct with only one phone connected. There's nowhere else to send it. With a second phone connected, this should read 1.

IF VER AND TTL LOOK LIKE NONSENSE

The packet format has probably changed since this guide was written. Open `BinaryProtocol.swift` in the bitchat repository, check the current header size and field order, and update the `OFF_*` constants at the top of the sketch to match. This is also why the UUIDs are in one clearly marked block — they're the other thing that can move.

The real test: two phones

Get two phones with bitchat, and separate them until they can no longer see each other directly. Place the relay between them. If they can now exchange messages, and your serial log shows `forwarded ... to 1 link(s)`, the relay is doing its job.

Once you're satisfied, set `STRICT_HEADER_LOGGING` to `0` and re-upload. It's very chatty and slows things down under load.

07 DEPLOY IT

Placement will beat every software tweak you could make.

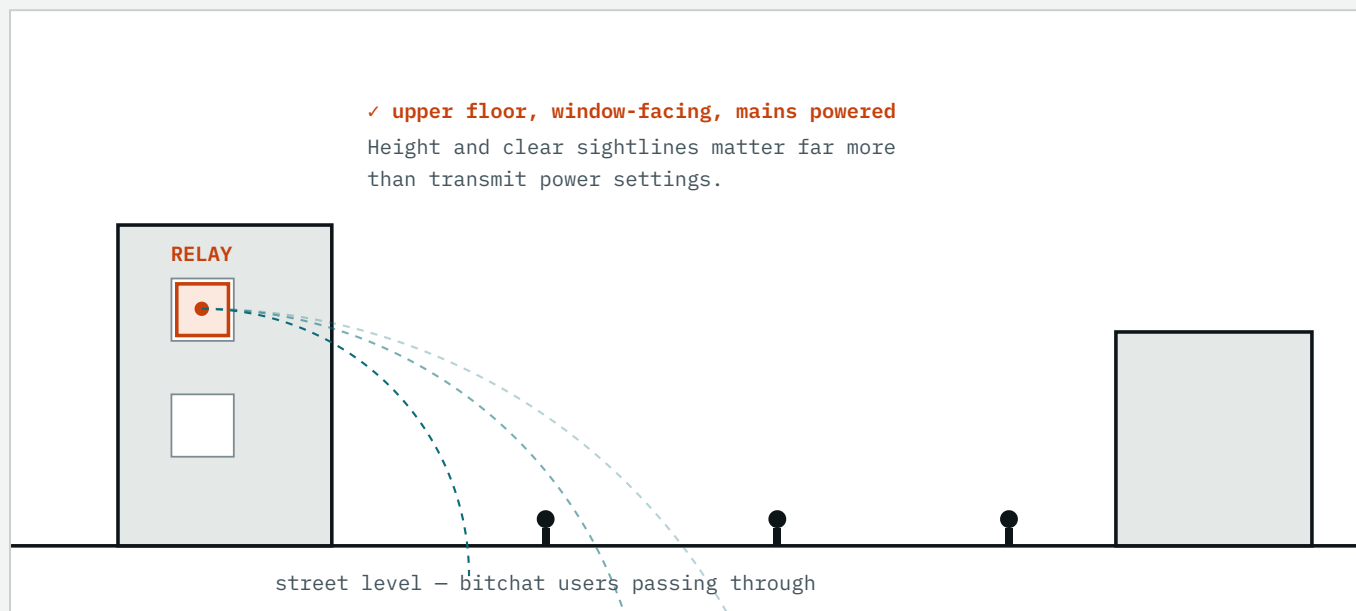


Figure 3 – Placement priorities, in order: height, clear line of sight to where people actually are, permanent power. A relay in a basement with a perfect antenna will lose to one in an upstairs window with the stock one.

PLACEMENT CHECKLIST

- ☐ As high as you can manage, with the antenna vertical.
- ☐ Near a window or exterior wall, not buried among furniture.
- ☐ Away from the Wi-Fi router, microwave, and any big metal object — all three are in or near the 2.4 GHz band and will hurt you.
- ☐ On permanent mains power, not a battery or power bank.
- ☐ Somewhere you can physically reach again to re-flash it later.

IF IT'S GOING OUTDOORS

- Put the board in an IP65 box; run the antenna *outside* the box through a cable gland. Sealing an antenna inside a plastic box is a smaller penalty than water ingress, but an external antenna is meaningfully better.

- Face the gland downward so water runs away from it.
- Add a silica gel packet inside to control condensation.
- Direct sun on a sealed dark box can cook an ESP32. Choose a light-coloured box or a shaded spot.

MORE RELAYS BEAT A BETTER RELAY

A mesh's usefulness scales with node count, not node quality. Two \$12 relays in two different windows will nearly always outperform one \$40 relay with a big antenna. Build the cheap one first, confirm it works, then repeat.

08 WHEN IT DOESN'T WORK

In rough order of how often each one is the actual cause.

SYMPTOM	LIKELY CAUSE	FIX
No serial port appears at all	Charge-only USB cable, or missing USB-to-serial driver	Swap the cable first. If that fails, install the CP210x or CH340 driver for your board's USB chip.
Failed to connect to ESP32	Board isn't in flashing mode	Hold BOOT during Connecting....., release once writing starts.
Serial monitor shows gibberish	Wrong baud rate	Set the dropdown to 115200.
Compiler errors about <code>onResult</code> / <code>setPower</code> / <code>NimBLEAdvertisedDeviceCal</code> <code>lbacks</code>	NimBLE-Arduino 1.x installed	Upgrade to 2.x in Library Manager.
Boots fine, <code>links=0</code> forever	No bitchat peers in range — the usual answer	Stand right next to it with the app open and foregrounded. Confirm Bluetooth is on and the app has permission.
Still <code>links=0</code> with a phone right next to it	Wrong service UUID (testnet vs production), or a protocol change	Confirm the UUID ends <code>...4B5C</code> for production. Cross-check against <code>BLEService.swift</code> in the repo.

SYMPTOM	LIKELY CAUSE	FIX
no free client slots in the log	Connection limit still at 3	Redo section 04, and fully restart the IDE afterwards.
Connects, then drops repeatedly	Weak signal, or power supply can't hold up under TX bursts	Move closer to test. Try a better-quality 5V supply — brownouts during transmit are a common cause of random reboots.
Packets received but ver / ttl are nonsense	Byte offsets no longer match the app	Check BinaryProtocol.swift upstream and update the OFF_* constants.
Heap number falls steadily in [stat]	Memory leak from repeated failed connections	Power-cycle. If it recurs, reduce scan aggressiveness by raising the scan interval.

READING THE LOG

TAG	MEANING
[srv]	Peripheral role — someone connected <i>to</i> the relay
[cli]	Central role — the relay connected <i>out</i> to someone
[scan]	A bitchat advertisement was spotted nearby
[link]	A connection was added to or removed from the relay table
[rx]	A raw packet arrived (only with header logging on)
[relay]	A packet was decremented and passed on
[stat]	15-second heartbeat: live link count and free memory

09 POWER, ANTENNAS AND THE LAW

Short version: what's in this guide is fine. Amplifiers are where it gets complicated.

2.4 GHz is an unlicensed ISM band, but "unlicensed" means "operate within the rules without applying", not "no rules". In the US that's FCC Part 15; most other countries have a

close equivalent.

- **Setting TX power to 9 dBm in software** — fine. That's the chip's normal rated maximum and the module is certified to run there.
- **Fitting a higher-gain antenna** — generally fine for hobby use, and by far the best value upgrade. Note that gain isn't free: a high-gain antenna squashes the radiation pattern flatter, which is usually what you want for street-level coverage but reduces coverage directly above and below.
- **Adding an external RF power amplifier** — this is where you can exceed legal EIRP limits, and where a certified module stops being certified. Check your local regulations before going down this path.
- **Deploying on property that isn't yours** — get permission. A permanent unattended radio device on someone else's building is a conversation worth having in advance.

ON THE SOFTWARE SIDE

bitchat is open source and its protocol is publicly documented, so building an interoperable relay is straightforward and legitimate. The relay carries other people's encrypted traffic and cannot read it — but it is still infrastructure other people rely on, so if you take it down, and others have come to depend on it, letting them know is the decent thing to do.

WHERE TO CHECK WHEN SOMETHING CHANGES

- **Protocol and packet format** — `bitchat/Protocols/BinaryProtocol.swift` and `WHITEPAPER.md` in [the bitchat repo](#).
- **Service and characteristic UUIDs** — `bitchat/Services/BLE/BLEService.swift`.
- **Relay and TTL policy** — `RelayController.swift` and `BLEFanoutSelector.swift`, if you want to match the real client's behaviour more precisely than this sketch does.
- **A second opinion on the wire format** — `dearabhin/bitchat-cli`, a Python implementation that's wire-compatible with the app. Useful for cross-checking your byte offsets against working code rather than docs.
- **NimBLE API** — h2zero.github.io/NimBLE-Arduino.

Build order, condensed: flash it on your desk → confirm real header bytes with one phone → confirm a relayed message with two phones → turn logging off → mount it high → build a second one.

Protocol details in this guide were taken from the bittchat repository and whitepaper, and the API from NimBLE-Arduino 2.5.0 documentation. Both projects move; verify against upstream before trusting a deployed node.